

{Chapter 44: Stream Processing}

Kafka is seen as a powerful message bus, but traditionally it did not support processing when inside Kafka.

In Kafka data is stored but lacking the ability to get processed inside the broker itself. It's waiting for a consumer to transform its data.

From Kafka 0.10.0 allowing full stream processing library.

We will explain how Kafka streams used to calculate moving avg of stock prices.

What is Stream Processing?

Data-Stream $\rightarrow$ event stream, is an abstraction representing unbounded dataset.

Examples-

Stream of

- 1) credit card transactions
- 2) stock trades
- 3) package deliveries
- 4) network events going through $\rightarrow$ network calls
- 5) email sent; moves in a game

Event streams are ordered → ① depositing money,
order should be maintained ② then withdrawing money

Immutable records → events should be immutable.

Event streams are replayable → For most businesses, this feature is very imp. They would want to see a raw stream of events that occurred months ago to try to debug an issue

Kafka streams provide this capability of replaying streams.

Types of Stream Processing

- ① Request-Response Model → 1 ^{event} ~~request~~ came ; system processes it and responds back. Blocking call.
- ② Batch Processing → cron schedule, polls events every interval.
- ③ Stream processing → non-blocking call, No waiting for response like in request-response model

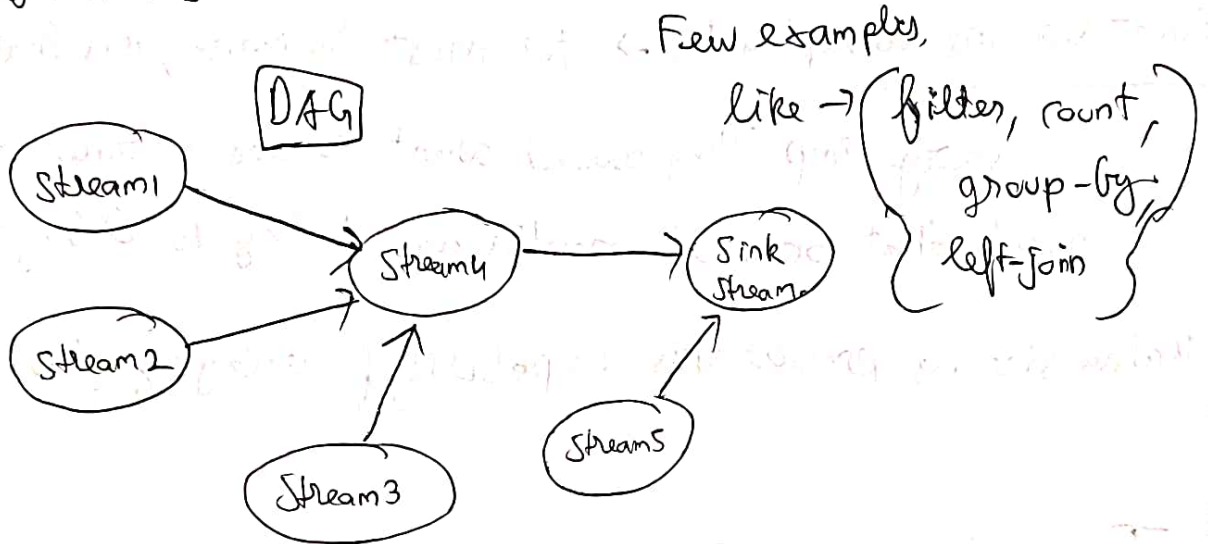
System keeps processing as events comes.

→ Used in alerting system, credit transactions etc

In a Stream few concepts are there

Topology

graph of streams join in a stream and populate at the end in a sink stream.



Time

Stream applications perform operations on time windows. For example, our stream might calculate moving 5-minute avg of stock prices. We need time as it's very critical when event comes etc

Event Time → User view of website timestamp (Event time usually the time which matters most)
→ most Imp metric

Log-Append time → Ingestion time, when it got persisted in Kafka. Not that Imp

Processing time → When the event is getting processed by the application
Two different threads can do different processing time
↓
Not reliable

As per the requirement of the developer, they can change the notion of timestamp they can use

- * When input record maps → output record directly then same timestamp as input can be used

- * When output is an aggregation, then we have to put the maximum timestamp out of all the events.

- * When output is result of joining two streams, timestamp output is the largest of two streams

- * Finally if output time is generated by KafkaStreams regardless of what input is → processing time

KafkaStreams provides an interface called TimestampExtractor.



Use any metric you want

- 1) Event time
- 2) log-Append time
- 3) Processing time.

Time zone Standardization should be there,

~~Note that~~ Suppose you are capturing clicks for a video all over the world, then ~~time~~ time zone conversion is imp

State.

Remember SMTP protocol for mailing.

In stream processing we do multiple operations like

- Counting number of events by type
- Moving averages
- joining two streams to create enriched stream.

So we need to keep track of info or a state, what was the previous picture → if you are string click count, you need to store how many clicks are already done.

It's often tempting to store this state in-memory or in a local hashmap, but not reliable as data in-memory is volatile and can be lost.

Local or internal state → in-memory or disk of the same machine.

Disadvantage → limited by the amount of memory available.
→ to leverage local or internal storage we have to focus on ways we can partition data, so that only small data is stored in each machine.

External State → State maintained in an external data store, NoSQL system like Cassandra,

Downside extra latency, cost to maintain centralized data store.

Stream Table Duality.

Two sides of the same coin,

- table tells you the current state of something
- stream tells you the history of changes that led to that state.

* You can build a table by replaying a stream of changes.

* You can create a stream by watching the changes happening in a table

Table → Its same as snapshot or current state,

Example

Customer Contact Table

this store
current
state

customerId	phoneNo.	emailId.
XYZ	~~~~~	~~~~~
ABC	~~~~~	~~~~~

Stream stores →

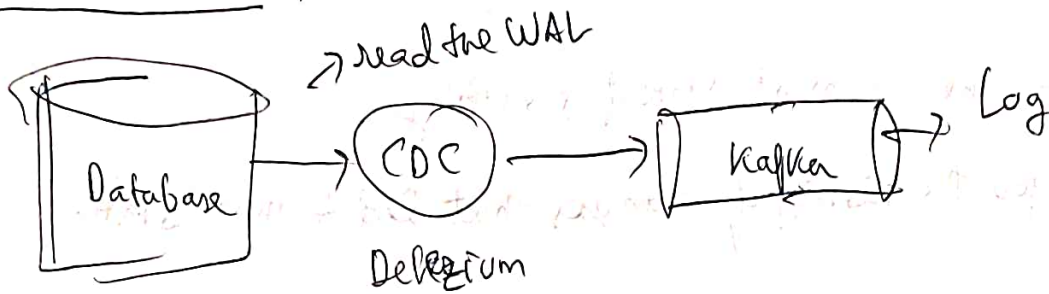
lets say customer XYZ changed his phoneNo, then emailId,
then again phoneNo.

So in log →

XYZ updated phoneNo - - ~
XYZ updated emailId - - ~
XYZ updated phoneNo - - -

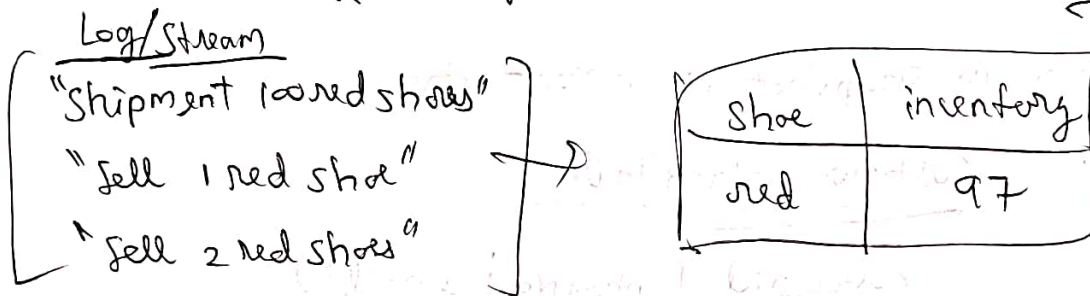
Converting b/w Stream & Table

Table $\rightarrow$ Stream $\Rightarrow$



Stream $\rightarrow$ Table $\Rightarrow$ (Materializing)

You can generate a table from a stream, it's called materialization.



Stream-Table Duality is powerful

It allows

- $\rightarrow$ Switch perspectives $\rightarrow$ State $\uparrow$ realtime vs full history $\uparrow$ Stream
- $\rightarrow$ Build flexible system $\rightarrow$ Conversion is easy
- $\rightarrow$ Recover from failure $\rightarrow$ even if state is lost

Time Windows → Important

Just like sliding window we use in ~~decoding~~ coding.
This can be also used in ad-click aggregator.

Windowed operations on streams helps operating on slices of time,

Join operations on two streams are also windowed - we join events that occurred at the same slice of time.

Size of window → (5 min, 15 min window)

Shorter the window → smaller the result

Larger the window → lag is more

Kafka streams also provide session window, where the size of the window is defined by period of inactivity.

Let's say a session window is there. If ~~session~~ no user session or activity is happening in the next 10 mins then close the window.

How often window moves. →

tumbling window → [0-5min, 5min-10min, 10min-15min]

~~sliding window~~

hopping window → [0-5] [1-6] [2-7] { overlapping window }

(Grace Period)
How long the window remains updated.

Let's say a window of $[0-5]$ min is opened.

Let's say we give a grace period of 3 min.

That means events upto 8 min will be allowed.

Any event which arrives after the window limit also has a ~~new~~ condition if we allow it or not.

$$(\text{event time} - \text{Upper limit of window}) > \text{grace period.}$$

~~Close the window.~~ (Reject the event)

We can use this in ad click Aggregator.

(Clock of the machine where window is created.)

→ we can use clock of global

Start from $[0-5]$ $[5-10]$

→ or we can use clock

when the
app boots
up

~~$[0-5]$~~ $[3:59 - 5:59]$

and so on

Stream Processing Design Patterns.

Some basic design patterns, which are known solutions to common requirements of stream processing architectures.

Single-Event Processing → Also known as map/filter pattern

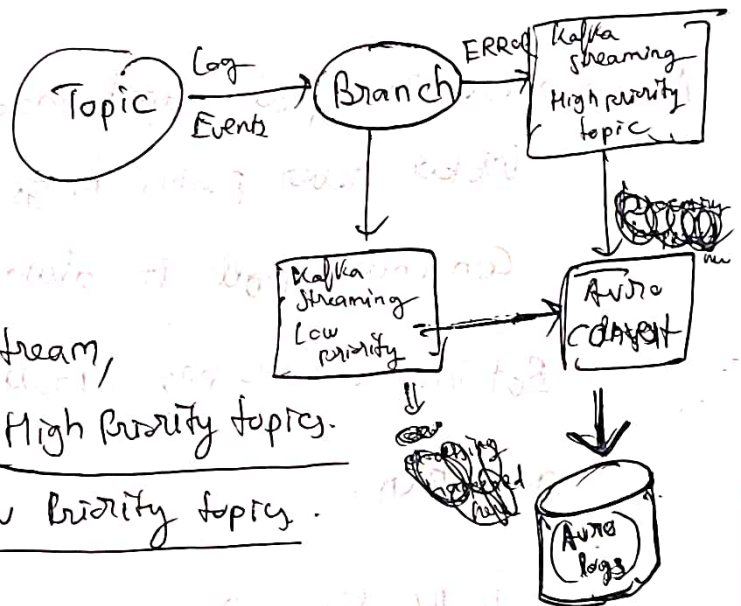
~~Simple~~ map coming from the Map/Reduce pattern.

map stage transforms events

reduce stage aggregates them

(Example 1)

App reads log messages from stream,
writes ERROR events into High Priority topics.
rest events go into Low priority topics.



(Example 2)

JSON format event $\xrightarrow{\text{Stream conversion}}$ AVRO format event

In that case we don't need to maintain state and it's easily recoverable

Seeing the above topology diagram

I asked chatgpt, where is the processing happening,
is it inside the broker itself or ~~as~~ a consumer does it

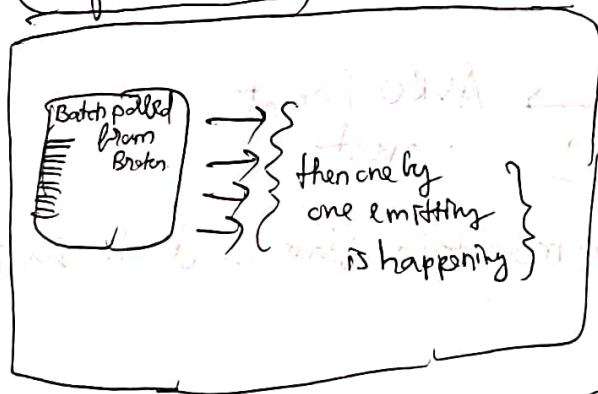
ChatGpt said, a special consumer does it,
called Kafka Streams, which does the processing

Follow up → So in Kafka Streams is it like a push based model,
does broker push messages to Consumer

Answer → No it's a pull based model only, ~~consumer never~~
broker never pushes msgs to ~~kafka~~ consumer.
Consumer polls it always.

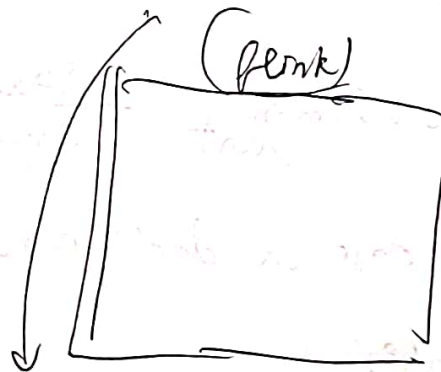
But interval is very small, so micro batching is happening
and then.

(Kafka Streams)



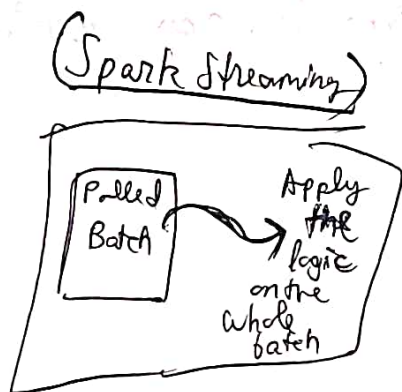
I asked gpt, does flink also do the same

(Flink)



So flink also does it like
Kafka consumers fetches in-memory the
batch and emits them one by one.

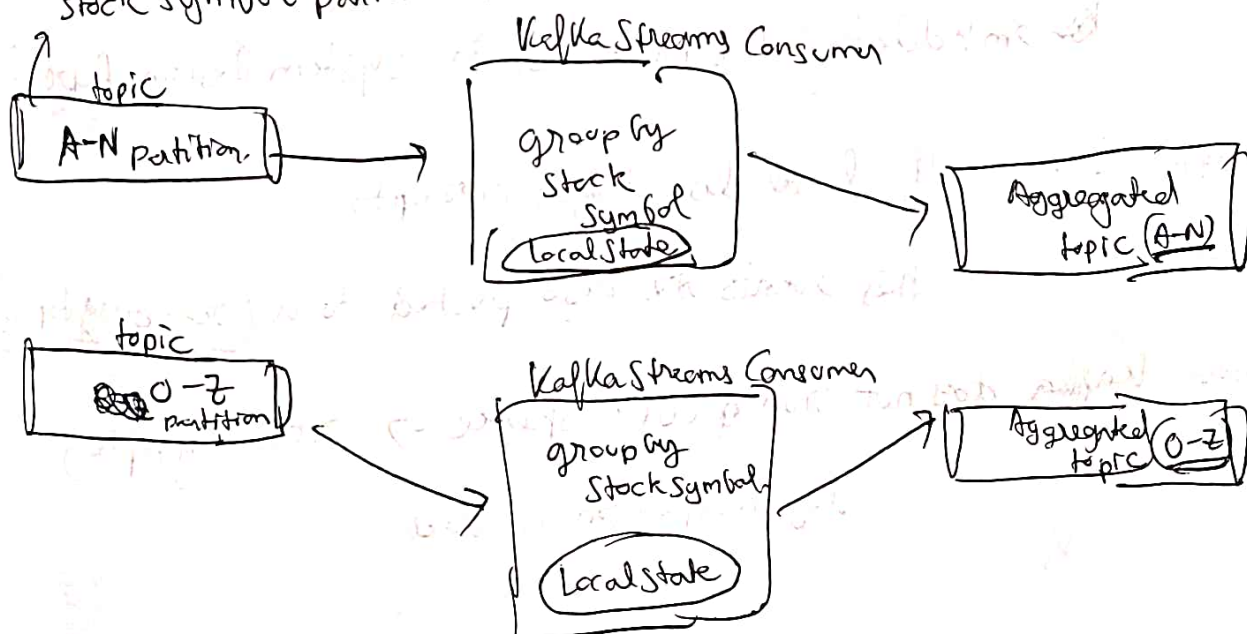
Then for doubt clearing, I asked does spark streaming also do the same, to which it replied No.



(Processing with Local State)

Most stream processing applications are concerned with aggregating information, especially window aggregation. An example of this is finding the minimum and maximum stock price for a day.

So local state to maintain on each machine, stock symbol partitioner.



Few issues with this pattern.

→ Memory usage

- * local state ideally fits into RAM, but some machines allow spilling to disk which can hamper performance, swapping and all those things.

→ Persistence

we don't want to lose state

To make sure that state is not lost when an application instance shuts down and state is recovered,

Kafka Streams handles it internally by using the embedded library of ~~Kafka Streams~~ RocksDB, so it's in the WAL of the machine (local disk)

RocksDB → has algo and infra exactly like Cassandra, but it uses the disk and RAM of the machine on which it is

embedded - { Algo mentioned in System Design Report }

To be extra safe, if local disk goes corrupt,

these events are also pushed to a (recovery topic)

Now Kafka does not run out of space → recovery topic,

log-compaction is used

Rebalancing case → let's say a partition which was assigned to one

consumer gets assigned to new consumer,

Then new consumer has to have the state retained back again,

It can do it using the recovery topic.

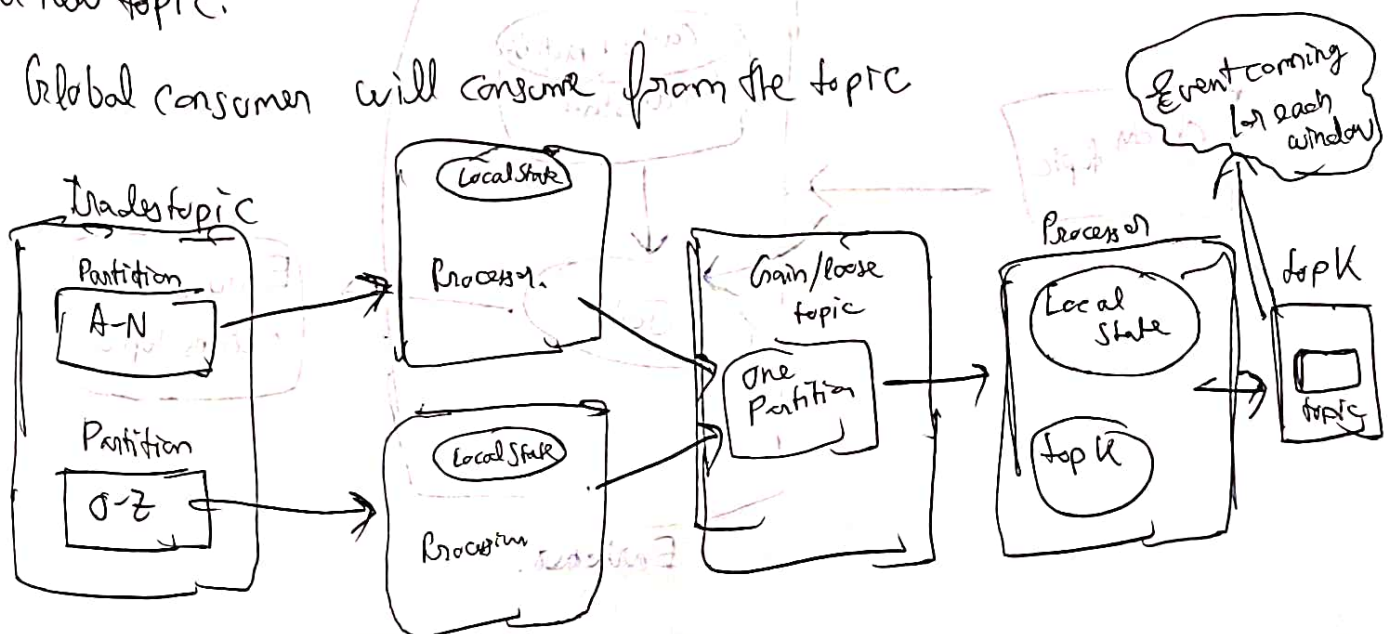
Multi-Phase Processing/Repartitioning

In the above example we were aggregating data per stock at each stock we are doing a group by Now requirement comes that you want to display top K stocks with gain.

In this case local state won't work. Because you need all details.

① So use the previous design to get details of each stock of a window, then ~~promit~~ emit these small info for each partition into a new topic.

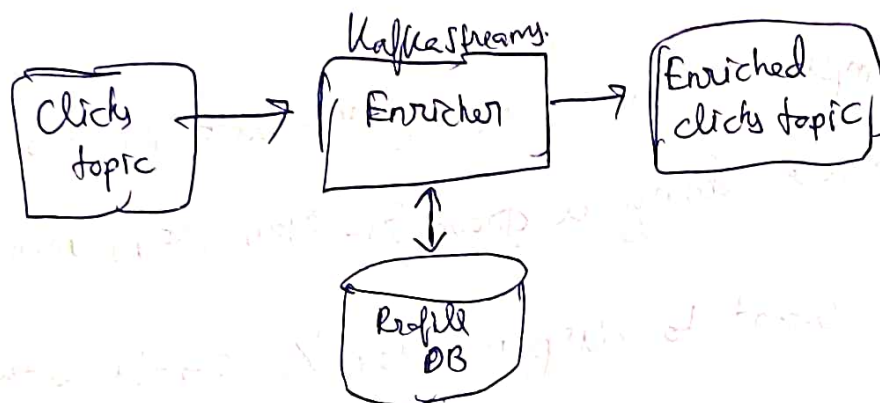
② Global consumer will consume from the topic



Processing with External Lookup : Stream-Table join

Often time stream processing integrations are required.
lets say we are getting a click event. Then that click is
~~not~~ enriched with user info like age & gender.

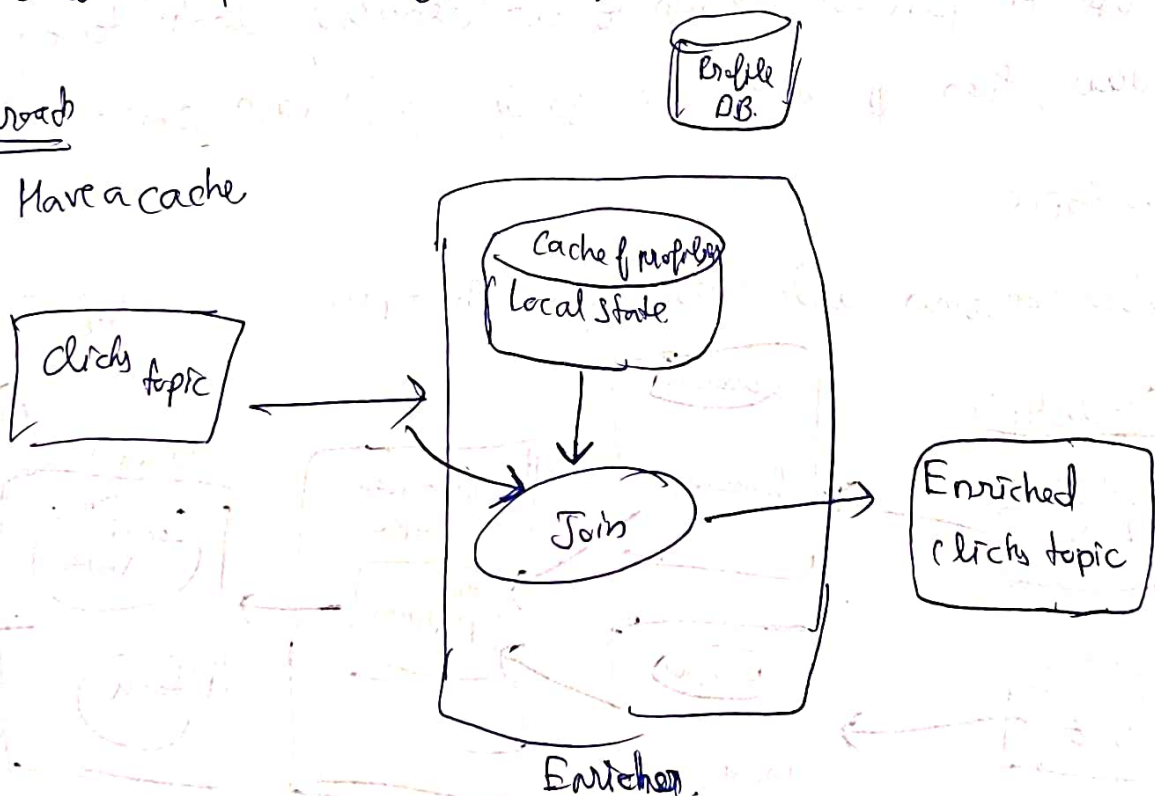
1st approach



But for each event if DB call is made then added 10-15ms latency,
stream processing can handle $(100-150)K$ events/s, but that much load
on DB is not possible. $DB \rightarrow 10K/s$

2nd approach

Have a cache



Now another edge case,

what if cache is stale. → Introduce CDC to update cache

